A Contract-Centered Architecture for Scalable and Manageable Agentic Runtimes

Working preprint | Agentic Runtime Research Project | 2026-07-22

1. Abstract

We model an agentic runtime as A=<S,H,X>: task Skills specify what may be done, a Harness compiles probabilistic intent into governed execution, and Scaffold instances provide isolated physical capacity. The central claim is that logical capability scaling (adding Skills) and physical throughput scaling (adding Scaffolds) are conditionally separable - they can evolve independently when the Harness completely mediates their interaction through explicit, typed contracts. This separation is conditional rather than axiomatic. It holds only when four assumptions are satisfied: Skills and Scaffolds meet through typed Harness contracts with operation closure; policy gates are deterministic; activated Skill paths are checked compositionally; and every binding is traceable and replayable. Violating any assumption breaks the separation on a specific axis, which we make explicit. We ground the extensibility thesis in a well-established engineering analogy - the three-axis decoupling of modern cloud applications - and derive a high-cohesion, loose-coupling context partitioning method from first principles of LLM inference. We use this model to derive concrete designs for external-data access, locality-aware parallel execution, planning-time dry-runs, and a Skill-as-Code lifecycle. Because the architecture has not yet been validated through a controlled implementation, we make no performance claims. Instead, we provide falsifiable protocols for capability pollution, physical scaling, Harness bypass, control-plane leakage, path-level safety, locality, and model-version stability. The intended contribution is a research agenda and an implementable vocabulary for scalable agentic runtimes whose growth remains observable, governable, and reversible.

2. Introduction

The practical problem in an agentic runtime is not simply how to make a large language model (LLM) call more tools. It is how to let a system gain capabilities and capacity without making either dimension opaque. A runtime may add a database Skill, a code-analysis Skill, and a procurement Skill while serving the same number of requests. It may also add workers, sandboxes, accelerators, or regional pools while exposing exactly the same capabilities. The first operation expands a logical action space; the second expands a physical execution space. Combining them in one undifferentiated agent loop hides the distinction that operations teams most need to see.

Consider a common scaling maneuver: every tool description, schema, policy, and example is loaded into every request so that the model can decide what to use. This appears flexible, but it turns capability registration into prompt growth. Unrelated Skills compete for attention, control metadata crosses into request context, and the effect of adding one capability is difficult to localize. A second maneuver adds more workers behind the same loop. Throughput may rise, yet safety and replay do not improve because the runtime still lacks a contract that explains what was planned, which checks ran, and why a particular worker was authorized. More capacity amplifies an execution model; it does not repair it.

This paper asks a narrower question than general agent intelligence: what runtime boundary permits logical capability scaling and physical throughput scaling to evolve independently while remaining manageable? Our answer is the Harness contract. Skill does not directly depend on Scaffold; Scaffold does not understand task-specific Skill semantics. Their dependencies meet only through typed Harness contracts. This is not an unconditional theorem. It is a design claim whose validity depends on mediation, type closure, deterministic gates, and trace identity.

The architecture is organized around a compact model:

$$A=<S,H,X>,$$

where S is a versioned set of Skills, H is the Harness that compiles and governs runs, and X is a pool of Scaffold instances. A Skill declares a capability contract. The Harness selects Skills, constructs an execution graph, applies policy and budget checks, binds graph nodes to Scaffold instances, and records evidence. A Scaffold supplies resources and isolation without interpreting the business meaning of the Skill.

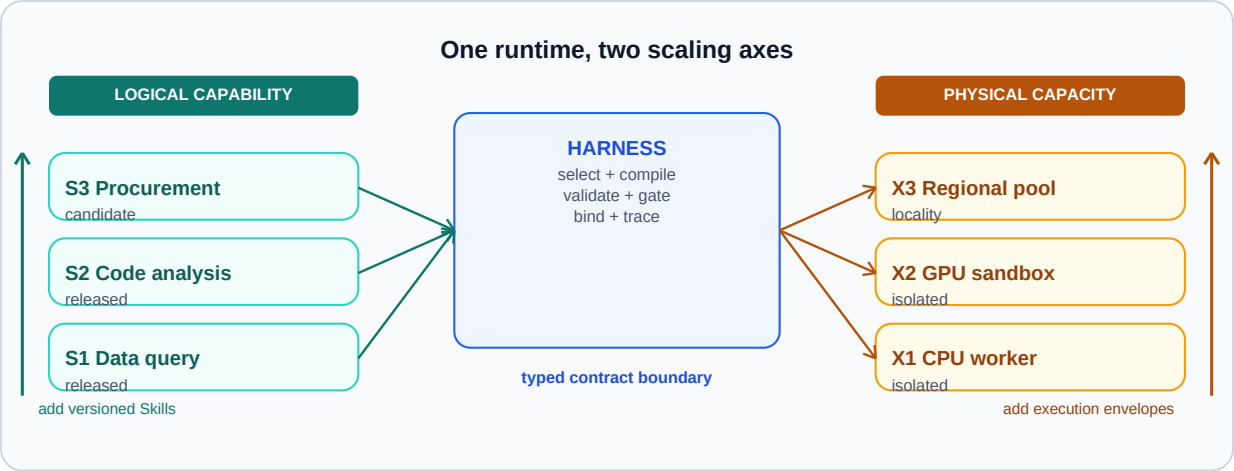

Figure 1. *Dual scaling model. Logical extension adds or revises Skills through the Harness contract; physical extension adds Scaffold capacity through the same binding boundary. Neither axis directly mutates the other, enabling capability growth and throughput growth to be evaluated as distinct interventions when Harness mediation is complete.*

Figure 1 establishes the paper's organizing distinction. Horizontal growth changes what the runtime can do; vertical growth changes how much governed work it can execute. The Harness is not merely glue between them. It is the control surface that makes both forms of growth inspectable.

2.1 Contributions

This paper makes four primary contributions.

1. We introduce the Skill-Harness-Scaffold model and state the conditions under which logical and physical scaling are separable in an agentic runtime.
2. We ground the extensibility thesis in the established engineering practice of three-axis cloud decoupling, and identify what is genuinely new when each layer contains probabilistic LLM calls.
3. We derive a high-cohesion, loose-coupling context partitioning method from first principles of LLM inference, showing that context organization - not model size - is the primary lever for agent performance.
4. We define the Harness as a manageable probabilistic control plane: planning may be model-generated, but activation, policy, budgets, binding, evidence obligations, and replay identities are enforced by typed and deterministic mechanisms.
5. We provide falsifiable evaluation protocols rather than synthetic results. The protocols isolate capability pollution, physical scaling, Harness bypass, control-plane leakage, composed-path safety, dry-run economics, and Skill stability across model versions.

The secondary contribution is architectural derivation. External-data access, parallelism and locality, planning-time dry-runs, and Skill-as-Code are not presented as unrelated features. Each follows from treating the Harness contract as the sole semantic-to-physical binding boundary.

3. Origins of the Extensibility Framework

3.1 The Cloud Application Analogy

The three-axis decoupling proposed here is not invented from scratch. Modern cloud applications have long operated with an implicit three-layer separation that maps directly onto our runtime objects. Virtual machines provide isolated execution capacity; software frameworks (application servers, message queues, API gateways) provide the runtime contract layer between business logic and infrastructure; and front-end applications compose user-facing capabilities on top. In this analogy, adding a new web page is a logical extension that does not require provisioning more virtual machines; adding more virtual machines is a physical extension that does not change which pages exist. The two operations are managed independently because the framework layer enforces a stable contract between them.

The Skill-Harness-Scaffold model is the agentic-runtime instance of this pattern. Skills correspond to front-end application code: they declare what the user-facing capability is, not how the machine runs. Scaffolds correspond to virtual machines: they provide isolated execution envelopes whose count scales throughput. The Harness corresponds to the framework runtime: it compiles declarative intent into executable units, enforces cross-cutting policy, and binds logical requests to physical resources. The mapping is not merely metaphorical; it predicts that the same decoupling properties - independent scaling, fault isolation, contract-based composition - should hold when the framework layer is well-defined.

3.2 What Is Genuinely New: Probabilistic Calls in Every Layer

The analogy also reveals the precise boundary of novelty. In a traditional cloud application, every layer executes deterministic code. In an agentic runtime, every layer may invoke an LLM: the Skills layer generates natural-language plans, the Harness performs contract translation and tool synthesis using model calls, and even the Scaffold layer's serving engine processes probabilistic inference. This creates a fundamentally new problem: the contract boundary between layers must mediate not just deterministic interfaces but probabilistic ones. A Skill declaration may be ambiguous; a Harness translation may vary across runs; a Scaffold's inference output is inherently stochastic.

This is the gap that existing work does not close. The policy-driven runtime layer of Zhang et al. [1] identifies the "seam" between agent frameworks (which understand semantics) and serving engines (which process events), and inserts an intermediate layer for cross-cutting policies. But its primitives - observe, score, predict, act - assume deterministic coordination of serving-level events. Our Harness contract goes further: it must mediate probabilistic intent (from Skills), probabilistic translation (within the Harness itself), and probabilistic execution (on Scaffolds), while producing deterministic gates and replayable traces. The core technical question is: under what conditions can a contract boundary between two probabilistic layers produce deterministic guarantees? The four assumptions of Section 4.3 are our answer.

3.3 Control-Plane and Data-Plane Orthogonality

A second source of the extensibility thesis comes from distributed systems theory. The separation of control plane (low-frequency decisions, configuration, routing) from data plane (high-frequency payload processing) is a well-established engineering law in SDN, Kubernetes, database query execution, and service meshes. In each domain, the separation enables independent optimization: the control plane can grow in complexity without proportionally increasing per-request data-plane cost, and data-plane throughput can scale without proportionally increasing control-plane overhead.

In an agentic runtime, this separation provides a second structural argument for decoupling. Logical scaling (adding Skills) primarily enlarges the control-plane decision space - new capability contracts, new selection rules, new composition invariants - without directly loading more tokens into the data plane. Physical scaling (adding Scaffolds) primarily enlarges the data-plane throughput capacity. Because the two forms of growth act on different planes, they are structurally orthogonal under the plane-separation invariant. Crucially, the agentic control plane is not the deterministic configuration distribution of SDN; it is a probabilistic control plane whose decisions may be LLM-generated. This introduces a novel correctness requirement: probabilistic control decisions must be bounded by deterministic invariants and recorded in auditable traces, because they cannot be trusted to be reproducible by default.

3.4 The Data Subsystem as a Third Axis

A third source emerges from the observation that agents do not only scale in capabilities and capacity; they also scale in the breadth of external data they must understand. An agent that starts with one data source may grow to access dozens: enterprise databases, SaaS platforms, personal files, and public internet sources. If each new source injects its schema, content, and metadata into the request context, data growth becomes a third form of prompt pollution. The data subsystem resolves this by pushing semantic summarization into an independent off-policy loop - an asynchronous background process with its own agents, its own model, and its own loop - that pre-summarizes sources into a queryable index. The main request path only consumes the query interface, not the full source content. This makes data scaling analogous to physical scaling: adding sources expands what is reachable without proportionally increasing per-request cost. Halevy et al. [2] empirically demonstrate the tension between pre-built metadata (high precision, low coverage) and on-line discovery (high coverage, low precision); the off-policy loop is designed to occupy the gap they leave open.

4. Related Work and Novelty Boundary

The closest work is a policy-driven runtime layer for coordinating agent frameworks and serving engines [1]. That line of work identifies an important systems boundary: policies chosen above the serving engine need runtime support below it. Our focus is complementary. We ask how task capabilities themselves are represented, activated, composed, and bound to isolated execution capacity. The distinction matters because scheduling policy can be correct while the capability path remains untyped or unaudited.

Jagarin [3] and Helium [4] explore runtime support for efficient agent execution. Dynamic runtime-graph work [5] similarly moves beyond static workflow templates by constructing graphs at run time. We adopt dynamic graphs inside the Harness, but use Scaffold in a narrower sense: execution and isolation infrastructure such as a sandbox, worker, container, accelerator allocation, or regional pool. It should not be confused with a workflow scaffold or prompt scaffold.

Tool Forge [6] treats tool production and adaptation as a systems problem. Skill-as-Code in this paper shares the asis on lifecycle, but its unit is a validated capability contract rather than only an executable tool. A Skill may include a tool adapter, a planner fragment, schemas, policy tags, tests, and postconditions. Five-plane runtime governance [7] provides a broader governance decomposition; our Harness can be understood as the point at which such governance planes become enforceable on a particular activated path. ClawVM [8] places compaction and writeback in a harness layer with deterministic and auditable guarantees, but its contract scope is narrower: it governs memory operations rather than the full capability-to-capacity binding.

Composition safety is especially relevant. Xie et al. show that components benign in isolation may become harmful in composition [9]. This observation directly challenges per-tool allowlists as a sufficient runtime defense. Our response is path-aware checking: the object of authorization is the activated Skill path plus its data and resource bindings, not a bag of individually approved Skills.

Production agent frameworks and orchestration engines occupy adjacent regions of the design space but do not expose the typed contract boundary studied here. AutoGen [10] coordinates multi-agent conversations but treats tool registration as prompt-level metadata rather than a versioned contract with deterministic gates. LangGraph [11] constructs stateful graph workflows but conflates planning and execution in a single runtime without a separate binding boundary to isolated capacity. CrewAI [12] assigns roles to agents but provides no path-level invariant checking or replay identity. The OpenAI Assistants API [13] bundles tools, context, and execution into one managed service where the contract between capability and capacity is implicit. Temporal [14] provides durable execution and replay for workflows but operates at the orchestration layer without agent-specific capability contracts or composition safety. AgentScope [15] and Agent-as-a-Service [16] offer multi-agent platforms and scalable infrastructure, respectively, but neither separates logical capability scaling from physical throughput scaling as distinct axes governed by a typed contract. Autellix [17] is the closest on the physical axis, modeling agents as general programs for serving throughput, but does not address the logical scaling axis.

Table 1 summarizes where existing systems fall relative to five Harness responsibilities.

Table. *Comparison of Harness responsibilities across production agent and workflow systems.*

System	Skill	Contract	Policy	Binding	Replay
	registration	compilation	gates	to capacity	/ audit
AutoGen	partial	no	no	no	no
LangGraph	partial	no	no	implicit	partial
CrewAI	partial	no	no	no	no
Assistants API	yes	no	implicit	implicit	no
Temporal	yes	partial	yes	yes	yes
Helium	no	no	no	partial	no
Tool Forge	yes	partial	partial	no	partial
Five-Plane	yes	no	yes	no	yes
This paper	yes	yes	yes	yes	yes

Prior agent-memory and retrieval-augmented generation (RAG) systems address context persistence and evidence retrieval [18, 19]. Those mechanisms can be hosted behind a Skill contract, but they do not by themselves establish the capability-to-capacity boundary studied here. SkillOpt [20] demonstrates that skills are the trainable external state of a frozen agent, with optimizer-level discipline applied in text space; our Skill-as-Code lifecycle extends this by adding a freeze-to-code stage that anchors determinism. CodeAct [21] shows that executable code actions improve agent composability; our work positions code not merely as an efficiency gain but as a deterministic anchor in a probabilistic control plane. The novelty claim is therefore deliberately limited: this paper does not propose a new model architecture, scheduler algorithm, memory representation, or universal governance taxonomy. It proposes a contract boundary and a set of conditions and tests for scalable, manageable agentic runtime behavior.

5. Model and Assumptions

5.1 The Three Runtime Objects

Skill.

A Skill s in S is a versioned declaration of task capability. Its minimum interface is

$$s = < g, i, o, p, e, v >,$$

where g describes the goal and applicability conditions, i and o are typed input and output schemas, p contains preconditions and policy tags, e declares externally visible effects, and v identifies the validated version. A Skill can include model instructions, tool adapters, graph fragments, examples, and tests, but these assets are implementation details behind the contract. A Skill is operation-closed when every external effect it can request is declared and testable through its interface.

Harness.

The Harness H is the runtime compiler and governor. Given a request q, identity and policy context u, and a Skill registry S, it produces a candidate execution graph D, validates the graph, binds accepted nodes to Scaffold instances, and emits a trace:

$$(q, u, S) \text{ -[H]-> } (D, C, b, tau).$$

Here C is the resolved Harness contract, b is a binding from graph nodes to Scaffold instances, and tau is the audit and replay trace. Selection and graph construction may use an LLM and are therefore probabilistic. Admission, schema validation, policy evaluation, budget enforcement, effect authorization, and trace creation must be deterministic for a fixed contract and policy snapshot.

Scaffold.

A Scaffold instance x in X is a physical execution envelope:

$$x = < r, z, l, a >,$$

where r describes available resources, z describes isolation and trust zone, l describes data and network locality, and a describes attestation and runtime identity. A Scaffold may be a process sandbox, container, virtual machine, browser session, graphics processing unit (GPU) allocation, or remote execution pool. It exposes resource facts and execution primitives, not task semantics.

5.2 The Harness Contract

The resolved contract for one run is

$$C = < I, O, G, B, E, T >.$$

I and O are the run-level typed inputs and outputs. G is the activated Skill graph. B contains resource, time, token, cost, and concurrency budgets. E contains authorized effects and evidence obligations. T contains policy, model, registry, data-snapshot, and trace identities needed for replay. This tuple is intentionally operational: each field must be serializable, inspectable, and independently rejectable.

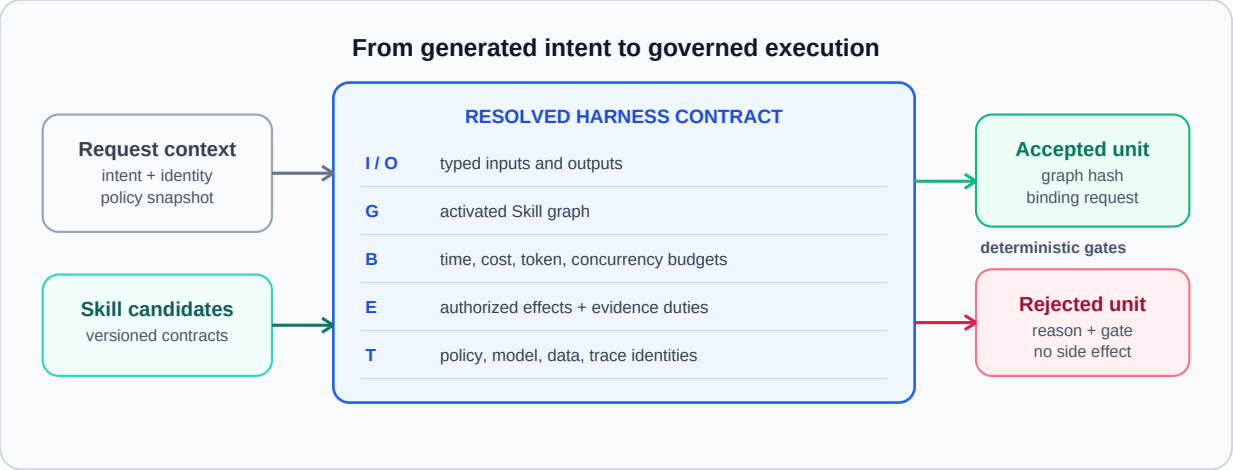

Figure 2. *Anatomy of a resolved Harness contract. A probabilistic plan becomes executable only after schemas, graph structure, budgets, effects, evidence obligations, and trace identities have passed deterministic gates. Manageability resides in the transition from generated intent to a typed, auditable execution unit.*

Figure 2 makes the Harness boundary concrete. A useful shorthand is: probabilistically written, deterministically and audibly executed. The phrase does not imply that execution outcomes are deterministic; external services and models remain variable. It means that the authorization decision, declared effects, selected versions, and evidence requirements can be reconstructed from the recorded contract.

5.3 Conditional Decoupling

Logical and physical scaling are conditionally separable under four assumptions.

1. Typed closure. Every activated Skill has typed inputs, outputs, declared effects, and a validated version; undeclared effects fail closed.
2. Complete mediation. A Skill cannot invoke a Scaffold or external system except through a Harness-issued binding.
3. Deterministic gates. Policy, budget, schema, and effect checks produce the same admission decision for the same recorded inputs and versions.

4. Trace identity. The run records enough identity to explain and replay planning, binding, policy, data snapshots, and effects within stated limits.

Under these assumptions, adding s_new to S need not change Scaffold semantics, and adding x_new to X need not change Skill semantics. The Harness contract is the stable interface. This is a falsifiable engineering claim, not an if-and-only-if theorem: hidden channels, undeclared effects, non-replayable services, or scheduler coupling can break the separation.

Conditional Separability. Under assumptions (1) - (4), DeltaS does not change Scaffold semantics and DeltaX does not change Skill semantics, provided no hidden channel, undeclared effect, non-replayable service, or scheduler coupling is introduced.

The converse is constructive: each assumption violation breaks the separation on a specific axis. Violating typed closure (1) allows a Skill to produce undeclared effects on a Scaffold, coupling logical changes to physical behavior. Violating complete mediation (2) lets a Skill bind directly to a Scaffold, so adding a Scaffold may require changing Skill semantics. Violating deterministic gates (3) makes admission decisions non-replayable, so logical scaling cannot be audited independently of physical state. Violating trace identity (4) removes the evidence needed to distinguish planning from execution failures, collapsing the two scaling axes into one opaque runtime. These inverse mappings are not proof of necessity in all possible systems - they are falsifiable predictions: if a system violates one assumption yet retains separability through some mechanism we have not identified, that system would refine our claim.

6. Harness Manageability

6.1 Control Plane and Data Plane

The control plane owns Skill registration, contract compilation, policy versions, placement rules, and release state. The data plane carries request payloads, executes accepted nodes, moves authorized evidence, and emits traces. The planes interact through references and resolved contracts rather than by copying the full control state into every model prompt.

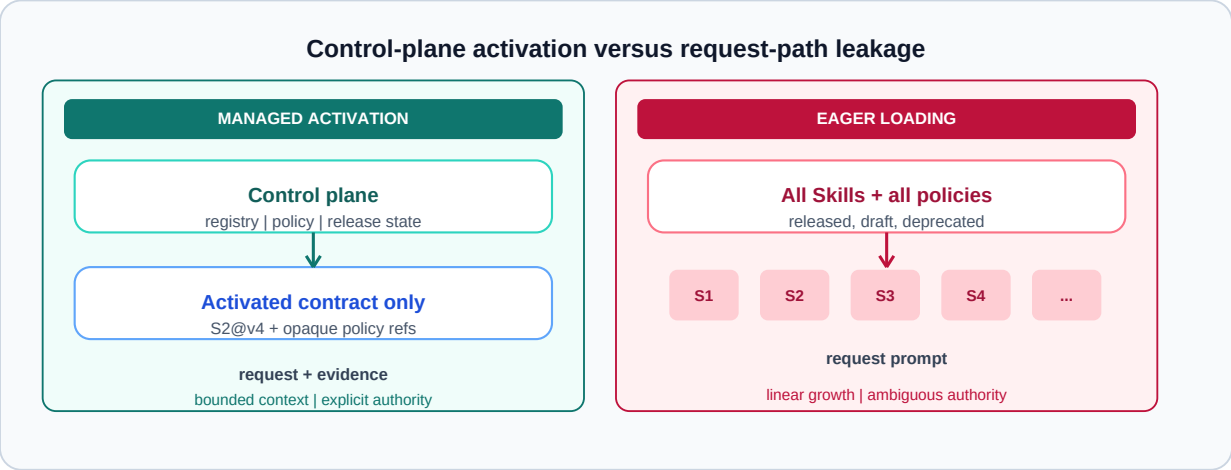

Figure 3. *Control-plane separation versus control-state leakage. In the managed design, the request receives only activated contracts and opaque version references. In the leaking design, every Skill schema and policy is injected into the request path. Selective activation prevents logical scaling from becoming linear prompt growth and limits exposure of governance metadata.*

Figure 3 highlights a failure mode that is easy to mistake for flexibility. Loading every Skill, application programming interface (API) schema, and policy example into each request leaks control-plane state into the data plane. Besides token cost, the leak creates ambiguous authority: a description visible to the planner may look executable even when its release or policy state has changed. The Harness should first select a small candidate set from registry metadata, then materialize only the accepted versions and constraints needed by the run. Tool Forge [6] demonstrates

that intent-scoped routing can reduce task-flow tool context by 99.2%, providing direct engineering evidence that selective activation prevents logical scaling from eroding data-plane throughput.

6.2 Path-Aware Invariants

Per-Skill approval is necessary but insufficient. Let G_q=(V_q,E_q) be the activated graph for request q, with nodes representing Skill operations and edges representing data or control dependencies. The Harness evaluates invariants over the path:

Accept(G_q)=AND over k phi_k(G_q,u,B,E,T).

Examples include separation of duties, data-residency continuity, prevention of read-then-publish paths, maximum cumulative privilege, and mandatory human approval before an irreversible effect. A path can violate an invariant even when each node is locally allowed. Runtime checks must also cover dynamically inserted nodes and retries, since they change the effective graph.

6.3 Audit, Replay, and Reversibility

An audit trace should distinguish three questions: what the planner proposed, what the deterministic gates accepted, and what the bound Scaffold actually executed. Conflating these stages makes incident review nearly impossible. At minimum, the trace records request and principal identity, candidate and activated Skills, contract versions, policy decision inputs, graph hash, binding decisions, external calls, evidence artifacts, and final effects.

Replay has levels. Decision replay recomputes gate outcomes from recorded inputs. Plan replay reruns planning under the recorded model and prompt version, accepting that stochastic output may differ. Execution replay re-executes against pinned data or service snapshots when available. The runtime should declare which level it supports rather than promising exact replay across mutable external systems.

Manageability also requires a stop path. Skill releases can be disabled, policy versions rolled back, bindings revoked, and in-flight graphs cancelled at Harness checkpoints. These mechanisms turn a trace from passive observability into operational control.

7. High Cohesion, Loose Coupling, and Context Partitioning

7.1 First Principle: The Agent's Job Is Organizing Context

A single LLM inference produces output determined by two components: the model's internal knowledge (weights, fixed within a task) and the context (variable input). Since the model weights cannot be changed at runtime, the only variable an agent loop can manipulate is the context. Planning, data retrieval, tool invocation, and memory writes are all, at bottom, decisions about what to place in the context for the next inference step.

This observation has a direct consequence for performance. KV-cache reuse in modern serving systems (vLLM, SGLang) depends on prefix stability: if the shared prefix is identical token-by-token, previously computed key-value tensors can be reused. If a single token changes in the middle of the prefix, all subsequent positions must be recomputed. The reusable part is not "the context that changed slightly" but "the context whose prefix is frozen token-by-token with changes only at the tail." An agent that jumps between heterogeneous tools or goals across iterations rewrites its context prefix every turn, collapsing cache hit rates and simultaneously diluting attention over irrelevant tokens. This is the mechanism behind the well-known degradation of long-horizon task control.

7.2 Context Partitioning by Tool-Set and Data Source

The design law follows: place the stable parts of the context (system prompt, tool-set schemas, fixed memory) at the front and freeze them token-by-token; place the per-turn task content at the tail. The longer and more stable the frozen prefix, the higher the cache hit rate. This is not merely about "keeping the context short" - it is about the structural layout of what goes where.

The primary axis for partitioning context is the tool-set. A sub-agent that repeatedly uses the same set of tools and harness contracts maintains a stable prefix across iterations, because the tool schemas, calling conventions, and policy tags injected into context do not change. Conversely, a sub-agent that switches between heterogeneous tool-sets rewrites its prefix every turn, destroying cache locality and diluting attention. This yields a concrete partitioning rule: divide sub-agents by non-overlapping tool-sets, so that each sub-agent's context prefix is maximally stable.

The secondary axis is data. Detail - raw tables, full document text, verbose intermediate results - should not flow between sub-agents as context. Instead, detail is written to the data subsystem (memory or lake), and only summarized context (conclusions, handles, key constraints) is passed forward. Downstream sub-agents retrieve detail on demand via semantic join, not by carrying it in their context. This reduces inter-agent communication from O(full context) to O(summary), raising orchestration efficiency and keeping each sub-agent's context focused.

7.3 Four Mechanisms for High Cohesion and Loose Coupling

The partitioning principles above yield four concrete design mechanisms that span all three runtime layers:

1. Tool-set cohesion (Skills layer). A sub-agent should use one stable tool-set throughout its lifecycle. This maximizes prefix stability and KV-cache hit rate. The anti-pattern is an agent that jumps between a database tool, a code tool, and a visualization tool in successive turns - each jump rewrites the prefix and invalidates cached computation.
2. Non-overlapping partitioning (parallelism). When a task is decomposed into parallel sub-agents, their tool-sets should be as disjoint as possible. Non-overlapping tool-sets imply non-overlapping resource contention and non-overlapping permission scopes, which reduces the serial fraction s in Amdahl's law and raises the achievable parallel speedup.
3. Summarized-context handoff (loose coupling). Sub-agents pass only summarized conclusions and handles to memory, not full context. The receiving sub-agent retrieves detail on demand from the data subsystem. This creates a loosely-coupled sequential relationship: dependencies flow through thin summary interfaces, while detail is absorbed by the memory layer. The data subsystem sits outside the three-layer stack, so this absorption does not introduce coupling into the runtime core.
4. Seed-affinity placement (Scaffold layer). Each type of sub-agent has a derivation seed - an image with pre-warmed tools, dependencies, and cache. Same-seed sub-agents are co-located to reuse image layers, tool processes, and KV-cache hot regions (affinity); different-seed sub-agents are spread across resource domains to reduce contention (anti-affinity). This maximizes physical locality within a type and minimizes cross-type interference.

The four mechanisms work in concert: mechanisms 1 and 4 maximize intra-agent cohesion (stable prefix, warm cache), while mechanisms 2 and 3 minimize inter-agent coupling (non-overlapping tools, thin summary interfaces). The root lever is tool-set partitioning, which simultaneously defines the Skills-layer locality boundary, the parallelism boundary, and the Scaffold-seed boundary.

7.4 Implications for the Contract Architecture

These mechanisms are not external optimizations applied after the contract is designed; they follow from it. The Harness contract determines which tools a sub-agent may invoke (mechanism 1), which sub-agents are independent (mechanism 2), what summarized schema flows between them (mechanism 3), and which seed template they derive from (mechanism 4). The contract is the single design surface from which cohesion and coupling properties emerge. A planning-time dry-run - which resolves the contract before execution - can probe tool-set closure, estimate prefix

stability, and check resource availability before committing any external effect, making the partitioning decision itself auditable.

8. Architectural Consequences

8.1 External Data as a Contract Instantiation

External data should not be a privileged side channel into the agent. It is a Harness-contract instantiation with explicit authentication, authorization, schema, snapshot, provenance, freshness, residency, and evidence obligations. A data Skill may discover or query assets, but the resolved contract determines what can be fetched and what evidence must accompany the result.

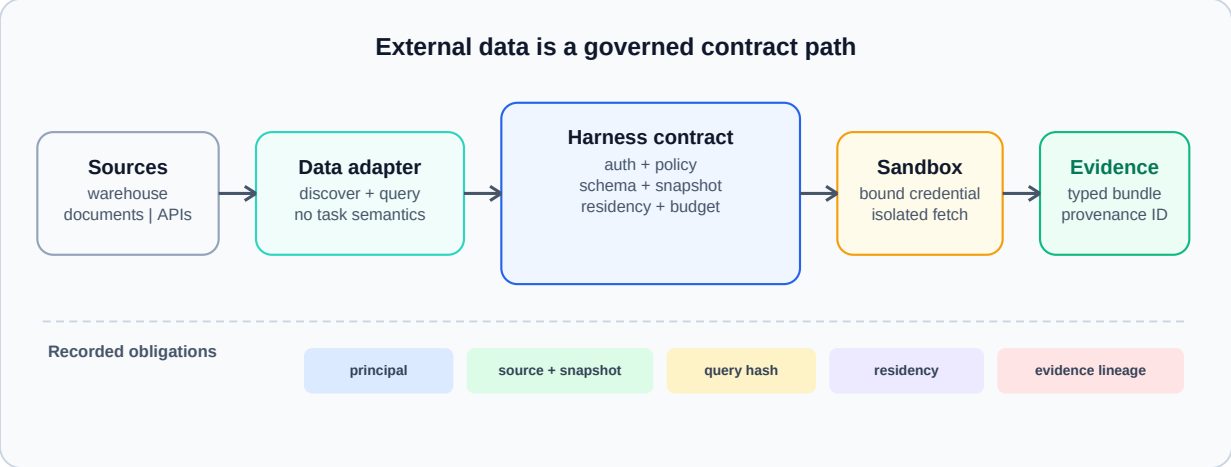

Figure 4. *External-data access as a Harness-contract instantiation. Source discovery and query planning remain semantic operations, while credentials, snapshots, provenance, residency, and evidence packaging are resolved and enforced at the contract boundary. Data governance composes with agent execution without embedding infrastructure credentials or source-specific policy in the Skill.*

Figure 4 shows how this boundary preserves both portability and accountability. The same analytical Skill can run against different warehouses or document stores when their adapters satisfy the contract. Conversely, the same data source can support multiple Skills without learning their task semantics. Evidence returned to the model is a typed bundle carrying source and snapshot identity, not an unqualified text fragment.

8.2 Parallelism, Locality, and Planning-Time Dry-Run

Once the Harness has a graph, it can expose parallelism that a monolithic prompt obscures. Independent nodes may execute concurrently; dependent nodes remain ordered by graph edges and effect constraints. The Scaffold pool contributes locality and capacity facts. The binding algorithm can then co-locate data-intensive nodes, isolate high-risk effects, and reserve scarce accelerators only for nodes that declare them.

A planning-time dry-run resolves graph structure, candidate bindings, budget envelopes, policy decisions, and expected evidence without committing external effects. It answers operational questions before execution: Which data regions will be crossed? Which nodes can run in parallel? Which irreversible effects require approval? Is the predicted cost within budget? Are all required Scaffold types available?

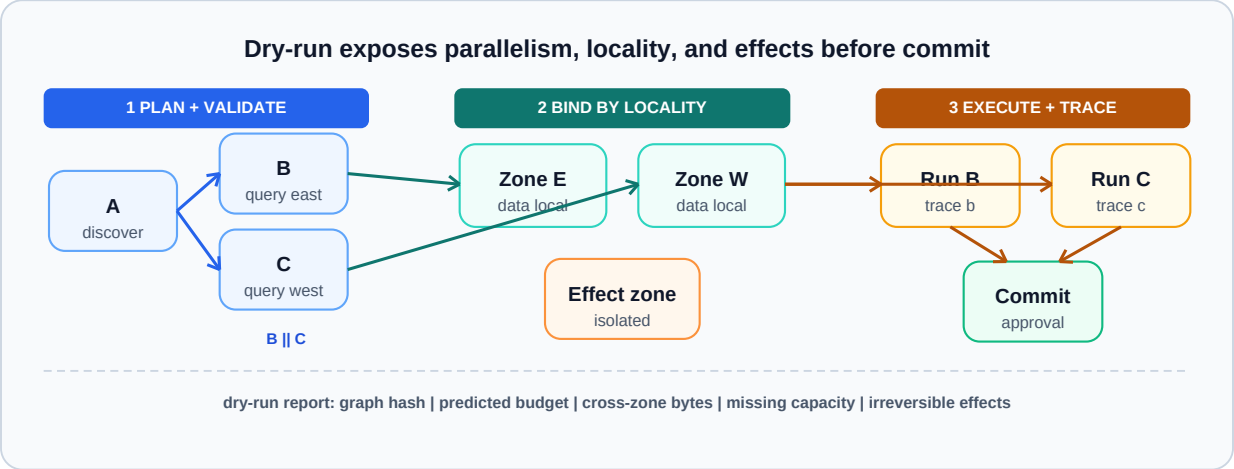

Figure 5. *Planning-time dry-run and locality-aware execution. The Harness first validates and annotates the graph, then groups independent nodes near required data and binds effects to appropriate isolation zones. Explicit graphs permit concurrency and locality optimization while preserving a reviewable pre-execution decision point.*

Figure 5 supports a modest claim: graph visibility creates an optimization opportunity; it does not guarantee a speedup. Dry-run adds planning overhead, and locality can conflict with load balance, policy, or accelerator availability. Evaluation must therefore report both avoided execution cost and dry-run cost.

8.3 Skill-as-Code

Treating a Skill as a prompt file makes capability growth difficult to review. Skill-as-Code applies a software lifecycle to the complete capability contract: source-controlled declarations, schema linting, effect review, contract tests, sandbox tests, model-version tests, signed release artifacts, staged rollout, monitoring, and rollback.

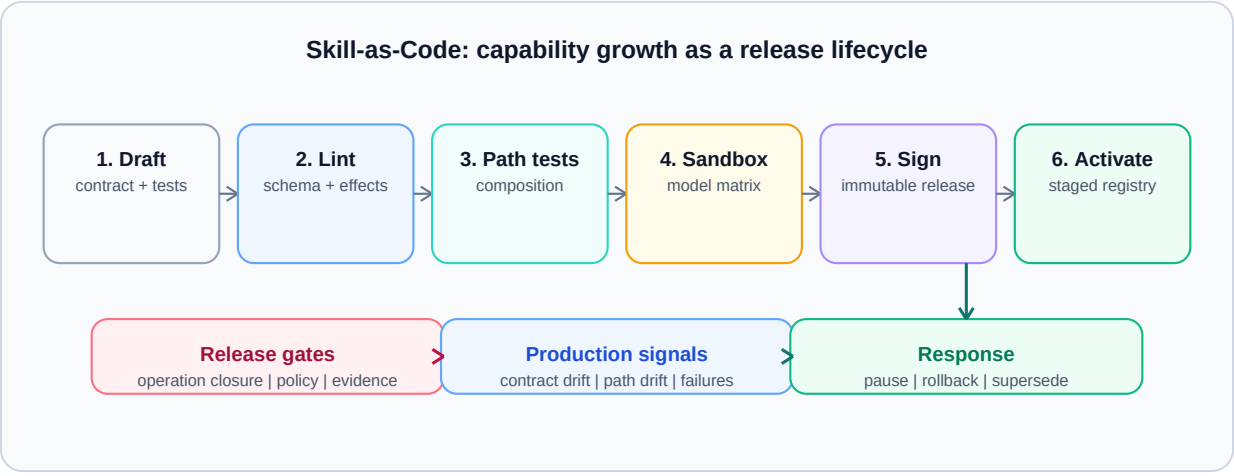

Figure 6. *Skill-as-Code lifecycle. A Skill becomes activatable only after contract linting, path and effect tests, sandbox execution, and a signed release; production traces feed monitoring and rollback. Logical scaling can be governed as a release process rather than as uncontrolled prompt accumulation.*

Figure 6 makes validation part of the architecture rather than a documentation convention. Operation closure is a release gate: tests must show that externally visible behavior is expressible through declared effects. Model-version stability is also tested at this boundary. A Skill can remain logically unchanged while its planner model changes, but the release should be blocked if contract adherence, path selection, or postcondition satisfaction regresses.

Skill-as-Code also serves as a deterministic anchor in the probabilistic control plane. When a Skill's operation space has closed - its tool-set is stable, its output format is structured, and its decision branches are enumerable - the execution process can be frozen into code rather than regenerated probabilistically each turn. This removes two sources of

uncertainty simultaneously: model sampling (code is a deterministic function) and context organization (the code embeds a stable procedure). The frozen prefix of Section 6.1 stabilizes what goes into context; skill-as-code stabilizes how decisions are made within it. Together, they turn the high-cohesion design law into an enforceable property: the stable prefix is frozen for cache locality, and the stable procedure is frozen as code for determinism.

9. Falsifiable Evaluation Protocols

The present work is architectural and has no controlled implementation results. We therefore specify experiments that can refute its claims. Each protocol separates the intervention from workload, model, policy, and hardware changes. Reported outcomes should include confidence intervals and failure distributions, not only averages.

Falsification matrix

Claim	Controlled intervention	Primary observations	Counts against claim
Logical scaling	Add unrelated Skills; freeze model and workload	context, selection error, task quality	interference grows with registry
Physical scaling	Add Scaffold instances; freeze Skills and policy	throughput, tail latency, decision consistency	semantics or gates change
Harness mediation	Introduce narrow bypasses	effects, evidence, replay	bypass has no control impact
Path safety	Compose locally allowed Skills	hazard recall, false positives	real hazards remain inexpressible
Dry-run + locality	Toggle dry-run and placement policy	overhead, avoided work, bytes moved	planning costs more than it saves
Skill-as-Code	Change model; freeze Skill source	contract and path stability	material drift passes tests

Figure 7. *Evaluation matrix for the architecture. Each row pairs one architectural claim with a controlled intervention, observable metrics, and a condition that would count against the claim. The architecture can be studied through separable, falsifiable experiments rather than through an omnibus benchmark score.*

Figure 7 summarizes the research program. The detailed protocols follow.

9.1 Logical Scaling and Capability Pollution

Hypothesis. Selective Skill activation through the Harness keeps request-context growth and task interference bounded as unrelated Skills are added.

Control. Hold workload, model, Scaffold pool, and active task Skills fixed. Compare eager prompt loading of all registered Skills with registry retrieval followed by typed activation. Add unrelated Skills in randomized batches.

Metrics. Measure activated-Skill precision and recall, prompt tokens, plan validity, tool-selection errors, task success, policy-decision latency, and accidental exposure of control metadata.

Expected signature. Under selective activation, registry size grows while activated context and task quality remain approximately stable; eager loading exhibits increasing context and more cross-Skill confusion.

Falsification. The claim is weakened if selective activation loses relevant Skills, if its retrieval overhead dominates, or if task interference grows at the same rate as eager loading.

9.2 Physical Scaling at Fixed Capability

Hypothesis. Adding compatible Scaffold instances increases sustainable governed throughput without changing Skill behavior or admission semantics.

Control. Freeze Skill, Harness, model, policy, and workload versions. Vary only the number and topology of Scaffold instances. Include homogeneous and locality-constrained pools.

Metrics. Measure throughput, queue time, tail latency, binding failures, isolation failures, cost per completed run, graph completion rate, and gate-decision consistency.

Expected signature. Queueing delay falls and throughput rises until a shared Harness, data, or external-service bottleneck is reached, while contract decisions remain invariant.

Falsification. The separation claim fails operationally if adding Scaffold capacity changes task semantics, bypasses gates, or yields negligible throughput because hidden centralized coupling dominates.

9.3 Harness Bypass Ablation

Hypothesis. Complete Harness mediation reduces unauthorized effects and improves decision replay.

Control. Execute the same workloads with full mediation and with narrowly instrumented bypasses: direct tool invocation, untyped data fetch, unrecorded retry, and direct Scaffold selection.

Metrics. Count undeclared effects, policy violations, evidence omissions, non-replayable decisions, incident localization time, and false rejections.

Expected signature. Bypass variants may reduce local latency but produce more unexplained or policy-inconsistent runs.

Falsification. If bypass does not measurably affect control or replay under adversarial workloads, the claimed value of the Harness boundary is overstated or the test lacks meaningful effects.

9.4 Control-Plane Leakage

Hypothesis. Passing only activated contracts prevents sensitive or irrelevant registry state from entering request context.

Control. Seed the registry with canary metadata, deprecated schemas, and policy text that is never required by the workload. Compare eager loading, retrieved activation, and opaque-reference activation.

Metrics. Track canary reproduction, prompt tokens, deprecated-tool selection, policy-version confusion, and plan accuracy.

Falsification. The claim fails if control metadata appears in outputs or influences plans despite opaque activation, or if opacity prevents the planner from satisfying valid requests.

9.5 Path-Level Composition Safety

Hypothesis. Graph-level invariant checks catch hazards that per-Skill approval misses.

Control. Construct Skill pairs and longer paths that are benign individually but violate data-flow, privilege, or effect-ordering constraints when composed. Compare local allowlists with path-aware admission.

Metrics. Measure hazardous-path detection, false positives, dynamic-graph coverage, and decision latency.

Falsification. The claim is weakened if invariants cannot express realistic hazards, if dynamic insertion routinely escapes checks, or if false positives make valid composition impractical.

9.6 Dry-Run and Locality Economics

Hypothesis. Planning-time dry-run and locality-aware binding reduce wasted work or data movement enough to justify their overhead for complex graphs.

Control. Hold graph and Scaffold pool fixed. Compare immediate execution with dry-run admission, and locality-oblivious with locality-aware binding.

Metrics. Report dry-run latency, avoided failed-run cost, bytes moved across zones, completion latency, resource utilization, and policy rejection stage.

Falsification. The feature is not justified where dry-run predictions are poorly calibrated, planning cost exceeds avoided work, or locality choices reduce utilization enough to increase total cost.

9.7 Skill-as-Code Across Model Versions

Hypothesis. Contract and path tests identify model upgrades that alter Skill behavior before production release.

Control. Freeze Skill source and test corpus while varying planner and executor model versions. Include perturbations of input phrasing, tool errors, and partial data.

Metrics. Measure schema adherence, declared-effect coverage, path stability, postcondition satisfaction, calibration of abstention, and test flakiness.

Falsification. The lifecycle is insufficient if production-significant changes pass the suite, or if nondeterminism makes the suite too unstable to gate releases.

10. Discussion

10.1 What the Architecture Does Not Solve

The Harness contract cannot make an unreliable model correct, an external service deterministic, or a malicious operating system trustworthy. It does not eliminate semantic ambiguity in Skill descriptions, and it does not prove that a policy captures an organization's intent. It can only make selected assumptions explicit and enforce the parts reducible to runtime checks.

The model also introduces a control-plane concentration risk. If the Harness compiler, registry, or policy evaluator is unavailable, execution may stop. If any is compromised, typed contracts can become a false assurance. Implementations therefore need replicated control services, signed artifacts, independent attestation, and a clearly scoped degraded mode. These are engineering requirements, not consequences guaranteed by the abstract model.

The cost of Harness mediation is not negligible and should be budgeted explicitly. Contract compilation - schema validation, graph construction, and invariant evaluation - adds a planning-time overhead that scales with the number of activated Skill nodes and the complexity of path invariants, not with registry size. In practice, this overhead is bounded by the activated graph, not the full Skill set: if a request activates 5 of 200 registered Skills, the gate evaluates 5 nodes and their composition edges, not 200. Based on analogous query-planning costs in database systems, we expect contract compilation latency in the low tens of milliseconds for moderate graphs (5 - 20 nodes), rising to hundreds of milliseconds for complex multi-step plans with cross-cutting invariants. Policy evaluation is cheaper still when rules are compiled to a decision cache indexed by contract hash. The critical design constraint is that this overhead must be amortized: for a request whose LLM inference takes seconds, tens of milliseconds of contract compilation is a small fraction; for a request that returns a single cached lookup, the same overhead dominates and indicates the request should bypass the full pipeline via a fast-path contract.

The degraded mode must be specified, not assumed. When the Harness compiler is unavailable, the runtime should reject new requests rather than fall back to unmediated execution, because unmediated execution breaks the assumptions of Proposition 1. When the policy evaluator is degraded, the runtime can admit only requests whose contracts are covered by a signed, cached policy decision with a valid freshness window. When the registry is unavailable, already-activated Skill versions remain usable for in-flight requests, but no new Skill versions can be activated. In all cases, the degraded mode narrows the set of admissible operations rather than silently widening them. The key invariant is: degradation reduces capability, never governance.

10.2 Trade-Offs

Selective activation can miss a relevant Skill. Strict operation closure can reject useful exploratory behavior. Path checking can be computationally expensive for dynamic graphs. Trace retention may conflict with privacy and data minimization. Replay may be impossible when external systems are mutable. Physical decoupling may be limited when a Skill requires specialized hardware or data residency. The architecture makes these couplings visible; it does not wish them away.

10.3 Research Questions

Several questions remain open. What is the smallest contract language that captures effects without becoming a second programming language? Which graph invariants can be checked statically, and which require runtime monitors? How should uncertainty from model-generated planning be represented in a deterministic admission decision? How can traces preserve enough evidence for audit while minimizing sensitive content? At what workload complexity does planning-time dry-run become economical? These questions define the next implementation phase.

11. Limitations and Threats to Validity

The paper presents a reference architecture, not a production system or empirical result. The model may underrepresent human approvals, long-running state, multi-tenant incentives, and cross-organization trust. The evaluation protocols could favor implementations that expose rich internal telemetry, making comparisons with opaque systems difficult. Synthetic composition hazards may not reflect real incident distributions; real workloads may contain confidential data that cannot be released.

Terminology is another threat. Skill, Harness, and Scaffold are overloaded across agent frameworks. We define them operationally, but mappings to existing systems will be imperfect. In particular, some frameworks combine Harness and Scaffold responsibilities in one service. The model can still be used analytically, but a clean implementation boundary may require refactoring.

Finally, conditional decoupling may hold only within a bounded operating region. A new Skill can require a new hardware class; a new Scaffold can expose timing or locality behavior that changes task quality. Experiments must record these couplings rather than classifying them away. A useful architecture should tell us where separation breaks.

12. Conclusion

Scalable agentic runtimes have two different growth problems: acquiring more capabilities and executing more governed work. The Skill-Harness-Scaffold model separates them by placing a typed, auditable contract between task semantics and physical execution. The central claim is conditional. Skill and Scaffold can evolve independently only when the Harness completely mediates their interaction, closes declared effects, applies deterministic gates to activated paths, and records identities sufficient for audit and bounded replay.

This framing changes the systems question from "how many tools can the agent see?" to "which capability path was activated, under which contract, on which resources, with which evidence and effects?" It also turns external data, locality, dry-run, and Skill lifecycle into consequences of one boundary rather than a list of platform features. The next

step is empirical: implement the boundary, run the falsification protocols, and report where the promised separation holds and where it fails.

References

[1] Zhang, Rui, Kim, Chaeeun, Hu, Liting. 2026. A Policy-Driven Runtime Layer for Agentic LLM Serving. arXiv preprint arXiv:2605.27744.

[2] Halevy, Alon, others. 2026. Do Agents Need Semantic Metadata?. arXiv preprint arXiv:2605.28787.

[3] Kadaboina, Ravi Kiran. 2026. Jagarin: A Three-Layer Architecture for Hibernating Personal Duty Agents on Mobile. arXiv preprint arXiv:2603.05069.

[4] Wadlom, Noppanat, Shen, Junyi, Lu, Yao. 2026. Efficient LLM Serving for Agentic Workflows: A Data Systems Perspective. arXiv preprint arXiv:2603.16104.

[5] Yue, Ling, Bhandari, Kushal Raj, Ko, Ching-Yun, Patel, Dhaval, Lin, Shuxin, Zhou, Nianjun, Gao, Jianxi, Chen, Pin-Yu, Pan, Shaowu. 2026. From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for LLM Agents. arXiv preprint arXiv:2603.22386.

[6] Rao, Swanand. 2026. Tool Forge: A Validation-Carrying Toolchain for Governed Agentic Execution. arXiv preprint arXiv:2605.28000.

[7] Tallam, Krti. 2026. A Five-Plane Reference Architecture for Runtime Governance of Production AI Agents. arXiv preprint arXiv:2606.12320.

[8] ClawVM Team. 2026. ClawVM: Deterministic and Auditable Agentic Runtime Harness. arXiv preprint arXiv:2604.10352.

[9] Xie, Yi, Du, Jiawei, Cheng, Yu, Zhou, Jiuan, Yin, Zhaoxia. 2026. Benign in Isolation, Harmful in Composition: Security Risks in Agent Skill Ecosystems. arXiv preprint arXiv:2606.15242.

[10] Wu, Qingyun, Bansal, Gagan, Zhang, Jie, Wu, Yiran, Li, Beibin, Zhu, Erkang, Jiang, Li, Zhang, Xiaoyun, Zhang, Shaokun, Liu, Jiale, others. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv preprint arXiv:2308.08155.

[11] LangChain. 2024. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. https://github.com/langchain-ai/langgraph.

[12] CrewAI. 2024. CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents. https://github.com/joaomdmoura/crewAI.

[13] OpenAI. 2024. OpenAI Assistants API. https://platform.openai.com/docs/assistants/overview.

[14] Temporal Technologies. 2024. Temporal: Open Source Microservices Orchestration Engine. https://temporal.io.

[15] Gao, Dawei, others. 2024. AgentScope: A Comprehensive yet Lightweight Multi-Agent Platform. arXiv preprint arXiv:2402.14034.

[16] Agent-as-a-Service Team. 2025. Agent-as-a-Service: Scalable Infrastructure for Production AI Agents. arXiv preprint arXiv:2505.08446.

[17] Liu, Xuting, others. 2025. Autellix: Serving Engine for LLM Agents as General Programs. arXiv preprint arXiv:2502.13965.

[18] Packer, Charles, Wooders, Sarah, Lin, Kevin, Wooders, John, Raganato, Felice, Restom-Gonzalez, Renel, Patel, Aman, Narayan, Shishir G.. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560.

[19] Lewis, Patrick, Perez, Ethan, Piktus, Aleksandra, Petroni, Fabio, Piktus, Vladimir, Karpukhin, Vladimir, Goyal, Naman, K uttler, Heinrich, Lewis, Mike, Yih, Wen-tau, Rockt aschel, Tim, Riedel, Sebastian, Kiela, Douwe. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems.

[20] Wang, Yizheng, others. 2026. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv preprint arXiv:2605.23904.

[21] Wang, Xingyao, others. 2024. Executable Code Actions Elicit Better LLM Agents. arXiv preprint arXiv:2402.01030.